

Apunte Semana 10 Clase II Inteligencia Artificial Clase

del 30 de abril de 2026

Alexander Brenes Garita
Carnet: 2018191805
Curso: Inteligencia Artificial
I Semestre 2026

I. INTRODUCCIÓN

La clase del 30 de abril de 2026 correspondió a la continuación del tema de funciones de activación y a la introducción formal de las Redes Neuronales Convolucionales (CNN, por sus siglas en inglés). El profesor retomó los problemas derivados del uso de la función ReLU estándar y propuso soluciones más sofisticadas, para luego hacer la transición hacia las CNN, motivando su uso con las limitaciones de las redes *fully-connected* al procesar imágenes.

II. FUNCIONES DE ACTIVACIÓN AVANZADAS

A. Problema de Neuronas Muertas en ReLU

La función ReLU (*Rectified Linear Unit*) presenta un problema conocido como “neuronas muertas”: cuando el valor de activación es menor o igual a cero, la derivada es exactamente cero. Esto impide que el gradiente fluya hacia atrás durante la retropropagación, bloqueando el aprendizaje de esa neurona de forma permanente. Formalmente:

$$\text{ReLU}(x) = \max(0, x), \quad \frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \quad (1)$$

Cuando $x \leq 0$, la derivada es cero y la neurona deja de actualizar sus pesos.

B. Leaky ReLU

La primera solución propuesta es *Leaky ReLU*, que sustituye el cero de la parte negativa por una pendiente constante pequeña:

$$\text{Leaky ReLU}(x) = \begin{cases} x & x > 0 \\ 0.01 \cdot x & x \leq 0 \end{cases} \quad (2)$$

Esta pendiente (0.01) evita que la derivada sea cero, permitiendo que el gradiente fluya siempre. Sin embargo, el valor 0.01 es fijo y puede no ser óptimo para todos los problemas.

C. Parametric ReLU (PReLU)

Para resolver la limitación de Leaky ReLU, se introduce *Parametric ReLU* (PReLU), donde la pendiente α no es un hiperparámetro fijo sino un **parámetro aprendible**:

$$\text{PReLU}(x) = \begin{cases} x & x > 0 \\ \alpha \cdot x & x \leq 0 \end{cases} \quad (3)$$

El valor de α se aprende durante el entrenamiento mediante retropropagación, junto con los demás pesos de la red. Esto agrega un parámetro adicional por neurona, aumentando la complejidad del modelo, pero mejorando el rendimiento.

Recomendaciones del profesor sobre funciones de activación:

- Evitar sigmoid y tanh en capas ocultas debido al problema de *vanishing gradient* en sus extremos.
- Comenzar siempre con ReLU en capas ocultas como punto de partida.
- Si ReLU no es suficiente, probar Leaky ReLU o PReLU.
- Para regresión: no usar función de activación en la capa de salida.

D. Función Softmax y Cross-Entropy

Para problemas de clasificación multiclase, la capa de salida utiliza la función **Softmax**, que convierte los *logits* z_i en probabilidades:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (4)$$

donde K es el número de clases. La salida es un vector de probabilidades que suma 1, que se puede comparar directamente con la etiqueta real (representada como *one-hot vector*).

La función de pérdida asociada es la **Cross-Entropy**:

$$\mathcal{L} = - \sum_{i=1}^K y_i \log(\hat{y}_i) \quad (5)$$

Se usa e^x por la misma razón que en otros contextos (e.g., similitud coseno): es estrictamente creciente y facilita las transformaciones matemáticas durante la diferenciación.

III. MOTIVACIÓN PARA LAS REDES CONVOLUCIONALES

A. Limitaciones de las Redes Fully-Connected con Imágenes

Las redes completamente conectadas (FC) tratan cada píxel como una entrada independiente. Para una imagen de $32 \times 32 \times 3$ (como CIFAR-10):

$$\text{Parámetros solo en capa de entrada} = 32 \times 32 \times 3 = 3,072 \quad (6)$$

Para imágenes de 200×200 , solo la primera capa conexión tiene $\sim 120,000$ parámetros. Añadir capas ocultas lleva rápidamente a millones de parámetros, lo que provoca:

- Mayor tiempo de entrenamiento y más épocas requeridas.
- Mayor cantidad de datos necesaria.
- Sensibilidad a la posición del objeto en la imagen (si el patrón se desplaza, la red falla).
- Rendimiento inferior comparado con arquitecturas especializadas.

B. El Problema de la Invarianza Posicional

Si una imagen de un “3” está centrada, la red aprende esa secuencia específica de píxeles. Si el mismo “3” aparece desplazado, la red no lo reconoce. Para compensar, se usan técnicas de *data augmentation* (rotaciones, traslaciones, etc.), pero estas no resuelven el problema de fondo de la ineficiencia paramétrica.

IV. REDES NEURONALES CONVOLUCIONALES (CNN)

A. Arquitectura General

Una CNN organiza sus neuronas en tres dimensiones: ancho, largo y profundidad. Su arquitectura típica consiste en la repetición de tres tipos de capas:

- 1) **Convolutional Layer:** Extrae características locales de la imagen mediante kernels deslizantes.
- 2) **Pooling Layer:** Reduce la resolución espacial (down-sampling) para condensar la información.
- 3) **Fully-Connected Layer:** Clasifica usando el vector de características resultante.

Las dos primeras forman el bloque de **extracción de características**, y la tercera actúa como clasificador.

B. Capa Convolutiva

1) **Kernels y Mapas de Activación:** Un **kernel** (o filtro) es una pequeña matriz de pesos que se desliza sobre la imagen realizando productos punto en cada posición. Para una imagen en color (3 canales), el kernel tiene dimensión $K \times K \times C_{in}$, donde C_{in} es el número de canales de entrada.

El resultado de aplicar el kernel es un **mapa de activación** (*feature map*). Si se tienen N kernels distintos, la capa produce N mapas de activación apilados, que constituyen la profundidad de la capa siguiente.

2) **Pesos Compartidos:** A diferencia de las capas FC, en la convolución los mismos pesos del kernel se reutilizan en toda la imagen. Esto reduce drásticamente el número de parámetros. Por ejemplo, un filtro de $5 \times 5 \times 3$ tiene solo **75 parámetros**, independientemente del tamaño de la imagen.

3) **Stride:** El **stride** S indica cuántos píxeles se desplaza el kernel en cada paso. Un stride de 1 produce mapas grandes; un stride mayor reduce la resolución de salida. La dimensión resultante después de una convolución es:

$$D_{out} = \left\lfloor \frac{M - K + 2P}{S} \right\rfloor + 1 \quad (7)$$

donde M es el tamaño de la imagen de entrada, K el tamaño del kernel, P el padding y S el stride.

4) **Padding:** El **padding** P agrega píxeles (generalmente ceros) alrededor de la imagen de entrada para controlar el tamaño de la salida. Para preservar exactamente las dimensiones de la entrada (*same padding*):

$$P = \frac{K - 1}{2} \quad (8)$$

Sin padding (*valid padding*), la salida es más pequeña que la entrada, lo que puede dificultar el diseño de arquitecturas profundas.

5) **No-Linealidad:** Después de cada convolución se aplica una función de activación no lineal (generalmente ReLU). Esto es esencial para que la red pueda aprender representaciones no lineales:

$$\text{Salida} = \text{ReLU}(\mathbf{W} * \mathbf{X} + b) \quad (9)$$

donde $*$ denota la operación de convolución.

C. Campo Receptivo

El **campo receptivo** (*receptive field*) es la región de la imagen de entrada que influye en una neurona particular. En capas tempranas el campo receptivo es pequeño (detalles locales como bordes y colores); en capas profundas es mayor (características de alto nivel). Esto es análogo a cómo funciona el sistema visual de los mamíferos.

D. Tipos de Filtros

Los filtros no son arbitrarios: existen tipos bien definidos con efectos conocidos. El profesor mencionó como ejemplo el **filtro gaussiano**, que distribuye los pesos siguiendo una campana de Gauss (pesos altos en el centro, bajos en los bordes). Al aplicarse, produce una imagen *borrosa* que elimina ruido y resalta contornos generales.

En una CNN, en lugar de definir los filtros manualmente, **los pesos se aprenden automáticamente** de los datos. Al final del entrenamiento, los filtros extraen información específica: bordes, texturas, colores, formas, etc.

E. Capa de Pooling

La capa de pooling realiza un **downsampling** de los mapas de activación. El tipo más común es *Max Pooling*, que toma el valor máximo en cada ventana. Sus características:

- Reduce el ancho y largo de los mapas de activación (generalmente a la mitad).
- **No modifica** el número de canales.
- **No tiene parámetros aprendibles:** solo aplica una función fija.

Por ejemplo, una entrada de $32 \times 32 \times 12$ después de Max Pooling con ventana 2×2 resulta en $16 \times 16 \times 12$.

F. Capa Fully-Connected Final

Una vez que las capas convolucionales y de pooling han condensado la imagen en un vector de características compacto, este se conecta a una capa FC para la clasificación. Las ventajas de esta estrategia son:

- El vector de entrada a la FC es pequeño (e.g., $13 \times 13 \times 256 = 43,264$), frente a los $\sim 150,000$ parámetros de una FC directa sobre la imagen original.
- Mejora la eficiencia computacional y reduce el sobreajuste.

V. ALEXNET: UN HITO HISTÓRICO

A. Contexto y Relevancia

AlexNet fue presentada en 2012 y representó un punto de inflexión en el Deep Learning. Fue la primera arquitectura CNN en ganar el desafío ImageNet LSVRC con un margen considerable, demostrando la superioridad del aprendizaje profundo para visión por computadora. Se entrenó en GPUs (2 GPUs en paralelo), algo revolucionario para la época.

B. Arquitectura

AlexNet procesa imágenes de $224 \times 224 \times 3$. Sus principales parámetros:

TABLE I
PARÁMETROS PRINCIPALES DE ALEXNET (PRIMERA CAPA)

Parámetro	Valor
Tamaño de imagen entrada	$224 \times 224 \times 3$
Tamaño del kernel (capa 1)	11×11
Número de filtros (capa 1)	96
Stride (capa 1)	4
Padding (capa 1)	0
Mapa de activación resultante	$55 \times 55 \times 96$

Aplicando la fórmula:

$$D_{out} = \left\lfloor \frac{224 - 11 + 0}{4} \right\rfloor + 1 = 55 \quad (10)$$

Esto confirma que los números en las arquitecturas **no son aleatorios**, sino el resultado directo de las fórmulas de convolución y las decisiones de diseño. La red continúa con capas sucesivas de convolución, Max Pooling y ReLU, finalizando en un vector de 4096 dimensiones conectado a la capa FC de clasificación (1000 clases en ImageNet).

VI. APLICACIONES DE LAS CNN

Las redes convolucionales resuelven una amplia variedad de tareas de visión por computadora:

- **Clasificación de imágenes:** Asignar una etiqueta a una imagen completa.
- **Segmentación de objetos:** Identificar y separar distintos objetos en una imagen.
- **Segmentación de instancias:** Contar instancias del mismo objeto (e.g., contar ovejas en una fotografía aérea de una granja).
- **Detección de objetos:** Localizar objetos con bounding boxes.

- **Super-resolución:** Aumentar la calidad y resolución de imágenes.
- **Auto-encoders:** Compresión y reconstrucción de imágenes.

VII. TRANSFER LEARNING

Un concepto clave mencionado al final de la clase es el **Transfer Learning** (aprendizaje por transferencia). Las capas tempranas de una CNN entrenada en un dominio (e.g., ImageNet) aprenden características generales (bordes, texturas, colores) que son útiles para cualquier tarea visual. Por lo tanto:

- 1) Se toman los pesos de las capas tempranas de una red ya entrenada (e.g., AlexNet).
- 2) Se usan como punto de partida (*inicialización*) para entrenar una nueva red en un dominio diferente.
- 3) Solo se re-entrenan las capas finales (más específicas del dominio).

Esto reduce significativamente el tiempo de entrenamiento y la cantidad de datos necesaria, ya que no se aprende desde cero. Es una técnica muy poderosa, similar conceptualmente a la “destilación de conocimiento” (*knowledge distillation*).

VIII. TEMAS SOBRE EL PROYECTO

El profesor habla del proyecto sobre algunos consejos directamente aplicables al proyecto del curso:

- Usar filtros pequeños (3×3 o 5×5). Los filtros grandes (11×11) son menos eficientes en términos computacionales.
- El tamaño de imagen mínimo para el proyecto es 128×128 ; considerar el impacto en la dimensión de los feature maps.
- Usar ReLU como función de activación por defecto en capas ocultas.
- Al tener problemas de dimensiones durante la implementación, aplicar la fórmula $D_{out} = \lfloor (M - K + 2P) / S \rfloor + 1$ para verificar las dimensiones.
- Citar correctamente los papers utilizados (e.g., el paper de AlexNet de Krizhevsky et al.).
- La investigación de arquitecturas especializadas (para móviles, para alta eficiencia) es parte del modelo B del proyecto.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012, pp. 1097–1105.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org>
- [5] V. Nair and G. E. Hinton, “Rectified Linear Units Improve Restricted Boltzmann Machines,” in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.